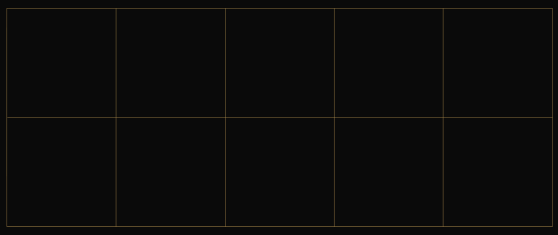
Command Menu
Context Card
Context Menu
Copy Button
Description
Destructive Action Modal
Destructive Confirmation Input
Dots Menu
Drawer
Empty State
Entity
Error
Error Card
Feedback
Fieldset
File Tree
Filter Params
Gauge
Grid
Input
JSON View
Keyboard Input
Label
Load More Button
Loading Dots
Menu
MiddleTruncate
Modal

Grid

Display elements in a grid layout.

Grid

A non-responsive grid with no cells.



Hide code

```tsx
import { Grid, GridSystem } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem debug guideWidth={1} unstable_useContainer>
      <Grid columns={5} height="preserve-aspect-ratio" rows={2} />
    </GridSystem>
  );
}
```

Basic grid

A non-responsive single grid with auto flowing cells configuration.

1	2	3
4	5	6

Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem guideWidth={1} unstable_useContainer>
      <Grid columns={3} rows={2}>
        <GridCell>1</GridCell>
        <GridCell>2</GridCell>
        <GridCell>3</GridCell>
        <GridCell>4</GridCell>
        <GridCell>5</GridCell>
        <GridCell>6</GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Solid cells

Using the `solid` prop on cells will occlude the guides that the cell overlaps

1 + 2		3
4	5 + 6	

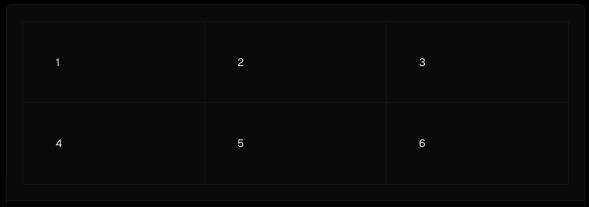
Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem guideWidth={1} unstable_useContainer>
      <Grid columns={3} rows={2}>
        <GridCell column="1/3" row="1" solid>
          1 + 2
        </GridCell>
        <GridCell>3</GridCell>
        <GridCell>4</GridCell>
        <GridCell column="2/4" row="2" solid>
          5 + 6
        </GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Responsive grid

Grid component with responsive `rows` and `columns` props at all 3 breakpoints.



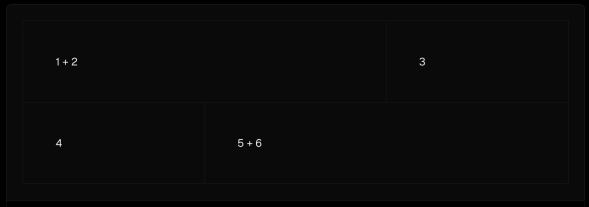
Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem unstable_useContainer>
      <Grid columns={{ sm: 1, md: 2, lg: 3 }} rows={{ sm: 6, md: 3, lg: 2 }}>
        <GridCell>1</GridCell>
        <GridCell>2</GridCell>
        <GridCell>3</GridCell>
        <GridCell>4</GridCell>
        <GridCell>5</GridCell>
        <GridCell>6</GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Responsive Grid with responsive guide clipping cells

Grid component with responsive `rows` and `columns` props at all 3 breakpoints as well as guide clipping on specific cells.



Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem unstable_useContainer>
      <Grid columns={{ sm: 1, md: 2, lg: 3 }} rows={{ sm: 6, md: 3, lg: 2 }}>
        <GridCell
          column={{ sm: '1', md: '1/3' }}
          row={{ sm: '1/3', md: 1 }}
          solid
        >
          1 + 2
        </GridCell>
        <GridCell>3</GridCell>
        <GridCell>4</GridCell>
        <GridCell
          column={{ sm: 1, md: '1/3', lg: '2/4' }}
          row={{ sm: '5/7', md: 3, lg: 2 }}
          solid
        >
          5 + 6
        </GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Grid with hidden row guides



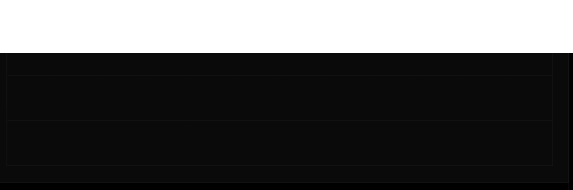
Hide code

```tsx
import { Grid, GridSystem } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem unstable_useContainer>
      <Grid
        columns={12}
        height="preserve-aspect-ratio"
        hideGuides="row"
        rows={3}
      />
    </GridSystem>
  );
}
```

Grid with hidden column guides





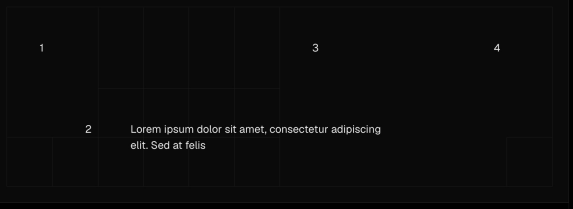
Hide code

```tsx
import { Grid, GridSystem } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem unstable_useContainer>
      <Grid
        columns={12}
        height="preserve-aspect-ratio"
        hideGuides="column"
        rows={3}
      />
    </GridSystem>
  );
}
```

Grid with overlaying cells

Grid component with cells that overlay another in various states.



Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem unstable_useContainer>
      <Grid columns={12} rows={3}>
        <GridCell column="1/3" row="1/3" solid>
          1
        </GridCell>
        <GridCell column="2/4" row="2/4">
          2
        </GridCell>
        <GridCell column="3/10" row="2/4">
          Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed at felis
        </GridCell>
        <GridCell column="7/12" row="1/-1" solid>
          3
        </GridCell>
        <GridCell column="11/13" row="1/3" solid>
          4
        </GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Specific Grid with Guide Clipping

Grid component with guide clipping enabled on specific cells.

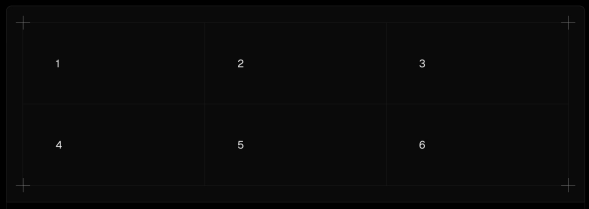


Hide code

```tsx
import { Grid, GridSystem, GridCell } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem guideWidth={1} unstable_useContainer>
      <Grid columns={3} rows={4}>
        <GridCell column="1/2" row="1/3" solid>
          1
        </GridCell>
        <GridCell column="3/4" row="1/2" solid>
          2
        </GridCell>
        <GridCell column="2/3" row="2/4">
          3
        </GridCell>
        <GridCell column="1/2" row="4/5" solid>
          4
        </GridCell>
        <GridCell column="3/4" row="3/5" solid>
          5
        </GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Grid with cross



Hide code

```tsx
import {
  Grid,
  GridCell,
  GridCross,
  GridSystem,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem guideWidth={1} unstable_useContainer>
      <Grid columns={3} rows={2}>
        <GridCross column={1} row={1} />
        <GridCross column={4} row={1} />
        <GridCross column={4} row={3} />
        <GridCross column={1} row={3} />
        <GridCell>1</GridCell>
        <GridCell>2</GridCell>
        <GridCell>3</GridCell>
        <GridCell>4</GridCell>
        <GridCell>5</GridCell>
        <GridCell>6</GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Dashed grid with cross

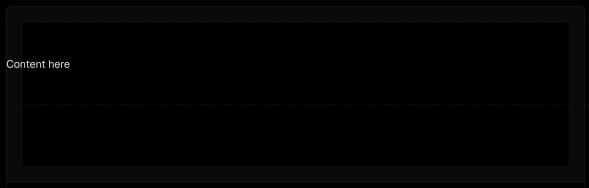


Hide code

```tsx
import {
  Grid,
  GridCell,
  GridCross,
  GridSystem,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridSystem dashedGuides guideWidth={1} unstable_useContainer>
      <Grid columns={1} rows={1}>
        <GridCross column={1} row={1} />
        <GridCross column={2} row={2} />
        <GridCross column={2} row={1} />
        <GridCross column={1} row={2} />
        <GridCell>Content here</GridCell>
      </Grid>
    </GridSystem>
  );
}
```

Dashed grid with grid page



Hide code

```tsx
import {
  Grid,
  GridCell,
  GridCross,
  GridPage,
  GridSystem,
} from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <GridPage>
      <GridSystem dashedGuides guideWidth={1}>
        <Grid columns={1} rows={1}>
          <GridCross column={1} row={1} />
          <GridCross column={2} row={2} />
          <GridCross column={2} row={1} />
          <GridCross column={1} row={2} />
          <GridCell>Content here</GridCell>
        </Grid>
      </GridSystem>
    </GridPage>
  );
}
```

Best Practices

When to use

- Use Grid for two-dimensional cell-and-guide layouts in marketing pages, docs landing pages, and feature breakdowns where the rule lines and cell borders are part of the design.
- For plain n-column responsive content (cards, lists), use Tailwind `grid grid-cols-*` utilities directly; Grid is overkill when guides aren't visible.
- Don't nest Grids more than one level; the guide overlap reads as visual noise and breaks the cell math.

Behavior

- Set `columns` and `rows` at all three breakpoints so cells reflow predictably between mobile, tablet, and desktop.
- Use `solid` cells to occlude guides behind a tile when content needs an opaque background; without it, guides render through the cell.
- Hide row or column guides only when their absence improves clarity (single-axis layouts, hero rows). Hiding both usually means a plain Tailwind grid is the right tool.

Accessibility

- Guides are decorative; mark them `aria-hidden="true"` and let semantics live on cell content.
- When cells become tappable, give each its own focus ring and keep tab order matching reading order.
- Confirm guide contrast on both themes; the default tokens are tuned, but custom borders can drop below the 3:1 minimum.